

ENGINEERING HANDOFF DOCUMENT

THE PLACEMENT ENGINE.

Every night, fifty-one team members. Twenty-two constrained slots. One deterministic engine that has to be right by the time the floor walks in.

SYSTEM

GLCR Grave
Deployment

COMPONENT

fill_engine.py · v8

AUDIENCE

Engineering Handoff

COMPILED

May 2026

TABLE OF CONTENTS.

A full read-through. The engine is greedy with a multi-objective scorecard, optionally swappable for a Hungarian solver on the constrained block. Everything below is in the order you'd want to learn it.

01	System Overview & Architecture	p. 03
02	Inputs – Roster, Profiles, Configs	p. 04
03	Schedule Parsing & Pool Extraction	p. 06
04	The Rule Registry	p. 07
05	Fill Order – The Daily Pipeline	p. 09
06	Skip Priority & The Z9 SR Specialist Rule	p. 10
07	Sweeper Auto-Assignment & Accommodations	p. 12
08	Hard Filters – Accommodations, Trainees, Back-to-Back	p. 13
09	The Scorecard – Multi-Objective Scoring	p. 14
10	Rotation Key & Sort Tuple	p. 16
11	The LAP Solver – Phase 4f	p. 17
12	Training Pre-Pass	p. 18
13	The Audit Pipeline	p. 19
14	Live-Update Mode	p. 20
15	Persistence – Workbook, Archive, Backend	p. 21
16	Configuration Reference	p. 22
17	Edge Cases & Guardrails	p. 23
18	Glossary	p. 24

HOW TO READ THIS

Sections 01-05 give you the lay of the land. Sections 06-11 are the meat — actual rules and code paths. Sections 12-18 are reference material. Code blocks reference real symbols in `fill_engine.py` and `glcr_engine/scorecard.py` ; line numbers may drift, but names are stable.

ARCHITECTURE.

The placement engine is the deterministic core of the GLCR grave deployment skill. Given a weekly schedule and a roster, it produces a complete seven-day deployment board, a full audit, an append-only archive, and a print-ready Zone Deployment Book.

WHERE IT LIVES

The engine is a Python module under [apps/zds/engine/](#) in the brijkillian-stack repo:

```
# apps/zds/engine/
fill_engine.py          # 2,171 lines — entry point + workflow orchestrator
render_deployment_book.py # 2,599 lines — print pipeline (separate concern)
glcr_engine/
  config.py             # SLOT_CATEGORY, SLOT_TO_AREA, ZONE_ADJACENCY
  scorecard.py          # multi-objective scorer (491 lines)
  lap_solver.py         # Hungarian solver, Phase 4f (263 lines)
  swap.py               # swap-chain proposer (Phase 2)
```

THE FIVE CONCERNS

The engine separates into five concerns; each is a discrete code path you can audit and reason about independently.

1 · INGEST

Read schedule workbook, roster, TM profiles, slot difficulty, training config, overlap tasks, utility porters. Build a normalized in-memory model.

2 · POOL EXTRACTION

Per calendar date, derive grave / PM-overlap / AM-overlap / personal-Monday pools from the schedule and apply roster eligibility filters.

3 · SOLVE

Daily fill pipeline: training pre-pass → restrooms → admin → Z9 SR → zones(skip-priority) → trash → support → overlaps. Greedy by default; LAP solver opt-in for the constrained block.

4 · PERSIST

Write to filled workbook, append placement archive, log to GLCR Memory Backend via [log_event](#), capture flags for callouts & [Z9_AREA_UNCOVERED](#).

5 · AUDIT

Cross-cutting, fires alongside every solve. Every placement records its scorecard breakdown; every unresolved slot, every override, every duplicate, every eligibility violation lands in the audit JSON and markdown. A polished board with no audit is unacceptable output.

OPERATING MODES

MODE	TRIGGER	OUTPUTS
<code>fill</code>	New TM schedule + template	Filled workbook, audit JSON+md, archive, deployment book, <code>log_event(kind=fill)</code>
<code>audit</code>	Filled workbook without refill	Audit JSON+md, <code>log_event(kind=audit)</code>
<code>reconcile</code>	Manual edits + post-edit check	Cell-level deviation report + repair recs + re-rendered book
<code>live-update</code>	Mid-week change in chat	Updated workbook + archive + book + downstream-impact analysis
<code>render</code>	"Print the sheets"	HTML + PDF only, no engine run
<code>archive-summary</code>	"What did we do for week X?"	Summary from archive xlsx or backend timeline

Default is `fill`. Modes are dispatched from the parent skill (`glcr-grave-deployment`) which owns the conversational layer; the engine itself is a deterministic CLI module that takes a schedule path and writes outputs. It is idempotent — re-running on the same inputs produces the same outputs.

Code refs: `fill_engine.py:1-80` (header docstring), `glcr-grave-deployment/SKILL.md` (modes dispatch).

ROSTER, PROFILES, CONFIGS.

The engine reads from one Excel workbook, one schedule workbook, and (since 5/5/26) Supabase. The legacy file-based config under `Rules/` is preserved for reference and emergency fallback only — the engine has been migrated to DB reads.

2.1 — ELIGIBILITY ROSTER

Source of truth for who can do what. 51 active TMs across three pools. The roster is the canonical TM identity ledger; if a TM isn't in the roster, the engine refuses to place them and emits `UNROSTERED_TM_PLACED` or `UNROSTERED_TM_IN_SCHEDULE`.

COL	FIELD	PURPOSE
A	Employee Name	Full legal name; matches schedule workbook
B	Display Name	Canonical board name. Only Abby has explicit value; others default to first word of A
C	Active	Y/N — N rows skipped
D	Grave Pool	<code>Grave</code> / <code>PM</code> / <code>AM</code>
F	Tie Break Rank	Integer 1-51. Lower = higher fill priority within pool
G-AF	Slot eligibility (Y/N)	Per-slot eligibility. <code>Y</code> = placeable, <code>N</code> = forbidden by roster
AG-AH	PM OL / AM OL eligibility	Y only for the corresponding overlap pool members

2.2 — TM PROFILES

Per-TM metadata loaded into the scorecard. Keys per TM:

SKILL_SCORE

Float 1-10 (0.5 increments allowed). Drives skill-stretch detection and skill-priority sort. `≤ 3` auto-classifies the TM as a trainee (locks them to MP / Support slots).

PREFERENCES[]

Array of `{target, stance, strength, note}`. `strength: hard` + `stance: avoid` filters the candidate from the pool; everything else is a soft scorecard adjustment via `preference_fit`.

ACCOMMODATIONS[]

Array of `{target, severity, status, note}`. `severity: hard` or `absolute` with `status: active` hard-blocks the TM from any matching slot. Targets accept exact slot labels, `area:Foo`, or `category:foo` (including the synthetic `category:sweeper`).

PAIR_AFFINITIES[]

Array of `{with, stance, note}`. Soft-bonus or soft-penalty when the partner is placed in an adjacent or same-area slot on the same date.

SCORE_HISTORY[]

Append-only observation log managed by the `glcr-tm-profiles` skill. Not used by the engine directly — used to derive trends.

LEGACY FIELD – PRESERVED

`slot_preference` from v1 is preserved at `metadata.legacy_slot_preference`. Values `restroom`, `no_sweeper`, `sweeper_capable` remain authoritative until fully migrated to `preferences[]` / `accommodations[]`. `no_sweeper` is currently set on Daryl only.

2.3 – SLOT DIFFICULTY

Per-slot integer 1-10 from `Rules/Slot Difficulty.json` (now: `get_slot_difficulty()`). Feeds the `skill_match` scorecard component. A placement is a "skill-stretch" when `(slot_difficulty - tm_skill) > 2` and is flagged as info-level in the audit.

2.4 – SLOT LOAD SCORES

Per-slot integer 1-10 representing physical & cognitive demand. Independent from difficulty. Feeds `fatigue_index` (a TM's load over the trailing 7 days) and `weekly_load_balance` (concentration within the current week). Used by the `avoid_high_load` override.

2.5 – TRAINING CONFIG

Per-date training day assignments: `{date: {trainee, trainer, day: 1-6}}`. Drives the training pre-pass (Section 12). Days 3 and 4 may be swapped for coverage.

2.6 – OVERLAP TASKS

Canonical task description per PM/AM overlap slot, with `_per_day_overrides` for one-offs. Read by both the engine (audit metadata) and the renderer (display).

2.7 – UTILITY PORTERS

Per-day utility porter assignments. Up to 8 entries per date. Currently populated manually; future schedules will include a Utility Porter row that the engine writes through directly.

Code refs: `fill_engine.py:75-95` (DB helper imports), `workbook-contract.md §1, §9`, `scorecard.py:30-80` (state shape).

SCHEDULE PARSING.

A weekly schedule arrives as an ADP/Kronos export with three sheets — Day, Swing, Grave. The parser turns row-per-shift records into four pools per calendar date: *grave*, *PM overlap*, *AM overlap*, and *personal Monday*.

3.1 — POOL DEFINITIONS

POOL	WORKER SHIFT DATE	PLACEMENT DATE	COVERAGE WINDOW
Grave	Date of grave shift	Same date	11 PM – 7 AM
PM Overlap	Date of PM shift	Same date	11 PM – 1 AM
AM Overlap	Date of 5 AM AM shift	Previous date	5 AM – 7 AM
Personal Monday	Off-cycle work week	TM's personal Monday	Per shift

3.2 — IDENTIFICATION HEURISTICS

The parser uses lightweight string matchers to classify a row's shift type. Two helpers (`fill_engine.py:760`) drive most decisions:

```
def is_5am(v): return v is not None and " 5:" in str(v)
def is_1am(v): return v is not None and "1:00A" in str(v)
```

A row with start time matching `is_5am` on the AM sheet becomes an AM overlap, placed in the previous day's row. A swing row whose end time matches `is_1am` qualifies as PM overlap (3 PM – 11 PM swing shifts do **not** qualify; only PM extending past 11 PM).

3.3 — THE PREVIOUS-DATE TRICK

This is the most error-prone part of the parser. AM overlaps are placed in the *previous* calendar row because their 5 AM shift overlaps the tail of the previous night's grave window.

WORKED EXAMPLE

Maria works AM Saturday at 5 AM. Her shift overlaps Friday's late grave window. Maria belongs in the **Friday** AM overlap pool, written into `dayFridayAMOL*` cells (row 13 of Week Overview). She is *not* placed on Saturday.

3.4 — PERSONAL WORK-WEEK MONDAY

Some TMs have a non-standard work week (e.g. Thursday–Wednesday). Their "Monday" lands mid-week. When detected, the engine fills their personal-Monday assignments on their personal Monday date. If the conflict produces a tie with a calendar-Monday worker for the same slot, the engine emits `PERSONAL_MONDAY_TIE_BREAK` warning rather than arbitrarily resolving it.

3.5 – HEADCOUNT RECONCILIATION

Once the parser finishes, every date carries a `scheduled_count` (unique TMs across all four pools). After the solve, `placed_count` is the unique TMs written to the board for that date. `delta = placed - scheduled`:

DELTA	MEANING	AUDIT
0	Clean – every scheduled TM placed	(none)
< 0	Some scheduled TMs not placed	<code>HEADCOUNT_MISMATCH</code> warning with names
> 0	More placed than scheduled – logic error	Critical – scan for duplicates / wrong-date pool

Code refs: `fill_engine.py:755-887` (parse helpers), `rules.md §2, §3`, `audit-spec.md §5`.

THE RULE REGISTRY.

The engine enforces a small, ranked set of rules. Hard rules filter the candidate pool before scoring. Soft rules are scorecard components — they bias placement but never veto. This page is the table of contents for those rules; each one gets a full treatment in later sections.

4.1 — HARD RULES (POOL FILTERS)

Applied in order inside `pick()` at `fill_engine.py:1180`. Each one removes candidates from contention *before* the scorecard ever runs.

RULE	EFFECT
Already placed today	Anywhere on the board this date — duplicate elimination is the first guard. Scope is the whole board, not the current section.
Trainee restriction	<code>skill_score ≤ 3</code> ⇒ Seth, Trenidee. Locked to MP / Support slots only. Skipped from <code>pick()</code> when <code>skip_trainees=True</code> .
Site-call avoidance	Per-call avoidance set passed by call-site (e.g. AVOID_PHYSICAL skips Z9 / Z9 SR / Trash for restroom-pref TMs).
Slot-preference avoidance	From <code>SLOT_AVOID_BY_TM</code> — Daryl no_sweeper, women-no-sweeper default, etc. Excludes specific (TM, slot) pairs.
Roster eligibility	<code>elig(rk, zone_name) = "Y"</code> . Anything <code>N</code> is forbidden.
Hard accommodation	<code>severity ∈ {hard, absolute}</code> + <code>status: active</code> + matching target. Filters before scoring.
Hard preference (avoid)	<code>strength: hard, stance: avoid</code> matching target. Filters with override fallback (see callout).
Back-to-back guard	For zones & RR sides — same exact slot as yesterday is rejected, with fallback if the slot would otherwise empty.

OVERRIDE FALLBACK

When a must-fill slot empties because every candidate is hard-blocked, the engine has a *graceful* override: it picks from the back-to-back fallback first, then from the hard-preference fallback (logged as `HARD_PREFERENCE_OVERRIDDEN` warning). It will never override an accommodation. **The engine prefers a noisy audit to an empty must-fill slot.**

4.2 — SOFT RULES (SCORECARD COMPONENTS)

Eight components combined as a weighted sum (lower = better). Full math is in Section 09; here's the punch list:

COMPONENT	DEFAULT WEIGHT	EFFECT
<code>skill_match</code>	1.0	Penalty when difficulty – skill > 2
<code>preference_fit</code>	1.5	Soft prefer / avoid from TM preferences
<code>pair_affinity</code>	1.0	Bonus / penalty per adjacent or same-area co-placement
<code>within_repeat</code>	1.0	Penalty if TM was in this exact slot earlier this week
<code>cross_week_rotation</code>	0.5	Bonus for slots not done recently (per archive, capped at 60 days)
<code>area_diversity</code>	0.7	Penalty if TM was in same floor area yesterday
<code>fatigue_index</code>	0.8	Penalty for stretched TMs in heavy-load slots
<code>soft_prefer_set</code>	0.6	Bonus if in per-call set (e.g. Admin rockstars)

4.3 – OVERRIDE CHANNELS

Three soft override types may be set per night (`set_soft_overrides_for_day`). Each is a small per-night nudge keyed by display name:

- **prefer_easier** – penalize this TM in slots where difficulty > 6.
- **avoid_high_load** – penalize this TM in slots where load > 6.
- **priority_placement** – bonus across all slots; "lean toward this TM."

4.4 – PHASE 4C EXPERIMENTAL COMPONENTS

Wired but default-zero-weight, awaiting tuning. Do not turn on without an A/B run:

- `sweeper_rotation_penalty` – rises with consecutive sweeper nights.
- `skill_stretch_reward` – small bonus when difficulty – skill = 1 (growth zone).
- `prior_run_continuity` – penalty for displacing a stable prior assignment (used in dry-run mode with `--config-override`).
- `weekly_load_balance` – penalizes concentrating heavy slots on one TM.

Code refs: `fill_engine.py:1180-1240` (pick), `scorecard.py:80-120` (DEFAULT_WEIGHTS).

FILL ORDER – THE DAILY PIPELINE.

For every calendar date in the week, the engine runs nine ordered passes. The order matters – most-constrained pools first so they don't get drained by less-constrained slots. **No blocking:** a slot being unfilled never prevents the next slot from being attempted.

01 TRAINING PRE-PASS	02 RESTROOMS	03 ADMIN	04 Z9 SR	05 ZONES (SKIP)	06 TRASH 1, 2	07 SUPPORT 1-3	08 PM OVERLAPS	09 AM OVERLAPS
----------------------------	-----------------	-------------	-------------	-----------------------	------------------	----------------------	----------------------	----------------------

WHY THIS ORDER

- **Restrooms first** – gender-locked = smallest pool. If they wait, zones consume the candidates and RRs go empty.
- **Admin second** – niche role with a soft rockstar preference (Sheri O, Sherry B, Jamie, Tawnya).
- **Z9 SR third** – specialist slot. Joy, Melissa, Mike S, Silvia get hard pick on Fri/Sat.
- **Zones fourth** – broad eligibility, plenty of candidates, can absorb skip-priority compression.
- **Trash + Support at the end** – floor-spanning overflow roles. Whatever pool remains lands here.

THE TWO DAY-TYPE SPLITS

Within the broad order above, two micro-orderings differ by day-type. They were tuned in v8 (5/1/26) to keep the M-Th board cleaner when fewer TMs are scheduled.

DAY TYPE	SPECIFIC ORDER (WITHIN FILL PIPELINE)
Fri – Sun	Admin → All Restrooms → Z9 → Z9 SR → Zones 1,2,3,4,5,6,7,8,10 → Trash → MP/Support
Mon – Thu	Admin → Restrooms (non-1+2) → Zones 1,2,3,4,5,8,9,10 → Restrooms 1+2 → Zones 6,7 → Trash → MP/Support

THE CONSTRAINED BLOCK

Zones 1, 4, 5, 8 + all 10 restrooms + Admin + Z9 SR form a 22-slot constrained block where eligibility is tightest and bad ordering most painful. With `placement_method="lap"`, this entire block is solved as a single Hungarian assignment instead of slot-by-slot greedy. See Section 11.

V8 OVERFLOW ROUTING – SUPPORT 3 & Z9 SR BUDDY

v8 added two conditional slots and a routing rule for un-placed grave-pool TMs after AM OL fill:

DAY TYPE	1ST OVERFLOW	2ND OVERFLOW
Fri / Sat	Z9 SR Buddy (col I)	Support 3 (col H)
Mon – Thu	Support 3 (col H)	—

Trainees are excluded from overflow — they should already have landed in their canonical Support 1 / Support 2 slot via the training pre-pass.

Code refs: [fill_engine.py](#) docstring (fill order summary), [fill_engine.py:1287](#) (training_prepass), [rules.md §6](#).

Z9 → Z3 AND THE SR SPECIALIST.

The engine never artificially limits candidates per slot, but pool exhaustion is real. When the grave pool runs short, zones empty in a deterministic order — not the order they appear on the board, and not first-come-first-served.

6.1 — THE SKIP LADDER

Brian's confirmed ladder (4/28/26 evening). Lower rank = skipped earlier; higher rank = most-protected:

SKIP RANK	ZONE	COVERED BY
1st (skip first)	Zone 9	Z9 SR + Zone 10 (both carry "And Zone 9" banners)
2nd	Zone 6	Restroom 6
3rd	Zone 7	Restroom 7
4th	Zone 2	Zone 4
5th	Zone 3	Zone 5
6th (most protected)	Zone 10	Restroom 10 (Z10 stays staffed longest)

Must-fill zones, never intentionally skipped: Z1, Z4, Z5, Z8.

6.2 — HOW THE ORDER PRODUCES THE LADDER

The ladder isn't enforced by special-casing each zone. It falls out naturally from the fill order: protected zones are filled *earlier* in the day-type-specific sequence, so they consume candidates first; skippable zones come later and absorb whatever pool remains. When the pool empties before reaching the late zones, those zones go un-filled and surface as `UNRESOLVED_SLOT` entries — not as critical errors, just as unresolveds.

6.3 — COVERAGE BANNERS

The renderer (`render_deployment_book.py:compute_alerts()`) inspects the fill state and emits an "And Zone N" banner on the covering position when its paired zone is empty. Banners are intentionally singular: when both Z9 SR and Z10 are staffed and Z9 is empty, both cards carry an independent "And Zone 9" banner — never combined into "And Zones 9 + 10."

Z9_AREA_UNCOVERED — CRITICAL AUDIT

If *both* Zone 9 and Z9 SR are empty after the fill loop completes, the smoke room area is operationally uncovered. The audit fires `Z9_AREA_UNCOVERED` with severity `critical`, surfaced in the audit markdown's `## ACTION REQUIRED` section, and captured to the GLCR Memory Backend as a flag with `kind: z9_uncovered`. The supervisor must manually escalate.

6.4 – Z9 SR SPECIALIST PREFERENCE (FRI / SAT ONLY)

Z9 SR is the smoke-room support position. It's the specialist slot — a small set of highly-trained TMs is preferred for it on weekends. The hard preference set:

```
ZONE9_WEEKEND_PREFERRED = {"Joy", "Melissa", "Mike S", "Silvia"}
ZONE9_PREF_DAYS          = {"Friday", "Saturday"}
```

On Fri/Sat, when filling Z9 SR, `pick()` sorts candidates by the scorecard then takes the first whose display name is in `prefer_names`. If none of the four are eligible / available, the standard scorecard pick wins. On Sun–Thu, the preference is dropped — anyone eligible should rotate through.

6.5 – WHY Z9 ITSELF HAS NO SPECIALIST PREFERENCE

Brian (4/28/26): Zone 9 the floor zone is very different work from Z9 SR. The engine deliberately does *not* apply specialist preference to Z9 — anyone with `Y` for Zone 9 can fill it on any day. Coupling the specialist set to both would over-concentrate the rotation on four TMs.

Code refs: `fill_engine.py` Zone 9 SR pick block, `render_deployment_book.py` `compute_alerts()`, `rules.md` §7, `audit-spec.md` §9.

SWEEPER AUTO-ASSIGNMENT.

Sweepers run two cross-floor routes — Sweeper 5/8/HL and Sweeper 9/10/SR. They are *not* a separate slot. Each shift, the engine auto-assigns one eligible male per route, sub-tasked onto an existing zone / restroom / trash card.

7.1 — ELIGIBILITY

- Must be **male** (women are excluded by default; override per-TM with `slot_preference: sweeper_capable` — currently nobody).
- Must be placed in zones, restrooms, or trash. **Not** Support, **not** Admin.
- Must not have `slot_preference: no_sweeper` — currently set on **Daryl** only.

7.2 — PRIORITY ORDER (EACH ROUTE ASSIGNED INDEPENDENTLY)

1. **Trash positions** — if the assigned trash TM is an eligible male, take them.
2. **Male in a zone the sweeper covers** — Z5/Z8 for Sweeper 5/8/HL; Z9/Z10 for Sweeper 9/10/SR.
3. **Any other eligible male** — stable order: zones 1-10, RRs (1+2, 6, 7, 8, 10), Z9 SR.

Each route is solved independently; the same person is never double-assigned. The renderer (`render_deployment_book.py`) owns the auto-assignment — the sweeper sub-task is appended to the assigned card's task list at print time, not stored as a separate placement.

7.3 — MANUAL OVERRIDE

`SWEEPER_REASSIGNMENTS` in `render_deployment_book.py`, keyed by ISO date, pins a sweeper to a chosen target slot for that day:

```
SWEEPER_REASSIGNMENTS = {
    "2026-04-29": {"sweeper_9_10_sr": "rr_8"},
}
```

7.4 — UNASSIGNED ROUTES

SWEEPER_UNASSIGNED — WARNING AUDIT

If no eligible male can be found (no trash male, no route-zone male, no other male in zones / RR), the route goes unassigned. Audit fires `SWEEPER_UNASSIGNED` with severity `warning`. Supervisor handles operationally.

7.5 – ACCOMMODATION MODELING

Sweepers and other physical-load constraints are modeled through accommodations (preferred path) or `slot_preference` (legacy). Three legacy values remain authoritative until migrated:

VALUE	EFFECT	CURRENTLY SET ON
<code>restroom</code>	Restroom-eligible TMs of any zone get first pick on RR slots; added to <code>AVOID_PHYSICAL</code> (skip Z9 / Z9 SR / Trash)	(rotating)
<code>no_sweeper</code>	Cannot do sweeper task or RR 1+2 (cross-floor route). Excluded from Z7, Z8, MRR1, WRR1	Daryl
<code>sweeper_capable</code>	A woman who CAN do the sweeper task; overrides default women-no-sweeper rule	(none)

7.6 – TRAINEE RESTRICTION

Trainee status is auto-derived from `skill_score ≤ 3` in TM Profiles. Currently:

```
TRAINEE_DISPLAY = {"Seth", "Trenidee"}
```

They are filtered from `pick()` when `skip_trainees=True` (the default for everything except MP / Support placement). Force-placing a trainee outside MP / Support fires `TRAINEE_VIOLATION` error.

Code refs: `render_deployment_book.py` sweeper assignment, `rules.md §9`, `audit-spec.md §9`.

THE CANDIDATE POOL FUNNEL.

Before a single scorecard component is computed, the candidate pool is funneled through a sequence of hard filters. Each filter takes a list of TMs and returns a smaller list. If the list ever empties, the engine falls through to its override chain.

8.1 — THE FUNNEL (`PICK()` AT `FILL_ENGINE.PY:1180`)

```
def pick(pool_list, placed_today, elig_col, zone_slot,
        prefer_elig=None, prefer_names=None, avoid_names=None,
        skip_trainees=True, skill_priority=False,
        soft_prefer_names=None, day=None):
    candidates = []
    fallback_btb = [] # back-to-back blocked
    fallback_hardpref = [] # hard-avoid preference blocked

    for rk in pool_list:
        dn = roster[rk]["display_name"]
        if dn in placed_today: continue # 1. duplicate
        if skip_trainees and dn in TRAINEE_DISPLAY: continue # 2. trainee
        if avoid_names and dn in avoid_names: continue # 3. site avoid
        if zone_slot in SLOT_AVOID_BY_TM.get(dn, ()): continue # 4. slot-pref
        if not elig(rk, elig_col): continue # 5. roster Y/N
        hard_block = has_hard_block(dn, zone_slot)
        if hard_block: # 6. accom + hard pref
            fallback_hardpref.append((rk, hard_block))
            continue
        if yest_iso and zone_slot in tm_slots_by_date.get(yest_iso, {}).get(dn, ()):
            fallback_btb.append(rk) # 7. back-to-back
            continue
        candidates.append(rk)
```

8.2 — HAS_HARD_BLOCK()

The function (in `scorecard.py`) returns the blocking entry if a TM can't be placed in this slot, or `None` if clear. It checks:

- Accommodations with `severity ∈ {hard, absolute}`, `status: active`, and a target matching this slot.
- Preferences with `strength: hard`, `stance: avoid`, and a target matching this slot.

Targets accept three forms: an exact slot label (`"Zone 8"`), an area prefix (`"area:Lobby"`), or a category prefix (`"category:restroom"`, `"category:sweeper"`, etc.). Category & area mappings live in `glcr_engine/config.py`.

8.3 — OVERRIDE CHAIN

When the candidate list empties, the engine refuses to leave the slot empty if a fallback exists. Lowest-cost fallback first:

1. **Back-to-back fallback** — TMs blocked only by yesterday's-same-slot guard. Pick from these without a separate audit; back-to-back is a hard guideline, not a rule.
2. **Hard-preference fallback** — TMs blocked only by a hard `avoid` preference. Pick from these and emit `HARD_PREFERENCE_OVERRIDDEN` warning per overridden TM.
3. **Accommodation fallback** — *never*. Accommodations are the only blocker the engine refuses to override.

DESIGN PRINCIPLE

The engine prefers a noisy audit to an empty must-fill slot. The override chain exists exactly so the supervisor sees overrides loudly the next morning, not so that slots quietly stay blank.

8.4 — PAIR-AFFINITY INTRA-DAY DUPLICATE GUARD

A defensive guard added 5/3/26 (`pair_affinity_score()`): if a TM has somehow ended up in a candidate pool for two slots on the same day, the function returns `999` for the second occurrence — a score so high they will never win the sort. The guard catches duplicate insertion paths that bypass `placed_today`.

Code refs: `fill_engine.py:1180-1240` (pick), `scorecard.py:has_hard_block`, `scorecard.py:pair_affinity_score`.

MULTI-OBJECTIVE SCORING.

After hard filters trim the pool, every remaining candidate is scored for the slot. The score is a weighted sum of eight to twelve components. Lower wins. Every component is bounded and has a documented role.

9.1 – THE FORMULA

$$\text{total} = \sum \text{weight}[k] \times \text{component}[k]$$

Where each component contributes a small positive value when it discourages a placement, a small negative when it encourages, and zero when neutral. The weighted sum is the placement cost.

9.2 – COMPONENT REFERENCE

COMPONENT	WEIGHT	RANGE	LOGIC
<code>skill_match</code>	1.0	0 / 1 - <code>skill/20</code> if <code>priority</code>	1 if <code>diff - skill > 2</code> , else 0. Skill-priority slots get an additional <code>-skill/20</code> nudge so high-skill TMs sort first.
<code>preference_fit</code>	1.5	-2 to +2	Sum over soft preferences with matching target: +1 per <code>avoid</code> , -1 per <code>prefer</code> .
<code>pair_affinity</code>	1.0	~-2 to +999	Walks all adjacent / same-area co-occupants on the day; for each, sums +1 avoid / -1 prefer in either direction. 999 if intra-day duplicate detected.
<code>within_repeat</code>	1.0	0 / 1	1 if TM was placed in this exact slot earlier in the current week.
<code>cross_week_rotation</code>	0.5	-1 to 0	<code>-min(days_ago, 60) / 60</code> . Recently-placed TMs sort later; never-placed TMs hit floor of -1.
<code>area_diversity</code>	0.7	0 / 1	1 if TM was in same floor area yesterday. Areas mapped in <code>SLOT_TO_AREA</code> (Lobby, Z6, Z7, Z8, Z9, Z10, etc.).
<code>fatigue_index</code>	0.8	0 to -5	<code>(7-day load / 8) × (slot_load / 5)</code> . Penalizes piling heavy slots on the same TM.
<code>soft_prefer_set</code>	0.6	0 / 1	0 if TM is in the per-call set (e.g. Admin rockstars), 1 otherwise.
<code>override_prefer_easier</code>	1.5	0 / 1	1 if "prefer_easier" override active on this TM and slot difficulty > 6.
<code>override_avoid_high_load</code>	1.5	0 / 1	1 if "avoid_high_load" override active and slot load > 6.
<code>override_priority_placement</code>	1.0	-1 / 0	-1 if "priority_placement" override active. Slot-agnostic — pulls TM toward any open slot.

9.3 – PHASE 4C EXPERIMENTAL COMPONENTS

Wired but default-zero-weight. Setting weights > 0 turns them on for an A/B run.

COMPONENT	LOGIC
<code>sweeper_rotation_penalty</code>	<code>min(recent_sweeper_nights / 3, 1)</code> for sweeper-tagged slots. Rises with consecutive sweeper nights.
<code>skill_stretch_reward</code>	<code>-0.5</code> bonus when <code>diff - skill = 1</code> (the "growth zone").
<code>prior_run_continuity</code>	<code>1.0</code> penalty when this candidate would displace a stable assignment from a prior run (dry-run mode only).
<code>weekly_load_balance</code>	<code>(week_load_so_far - 14) / 7</code> for slots with load ≥ 7 when the TM is already carrying ≥ 14 cumulative load.

9.4 – WHY A MULTI-OBJECTIVE SCORECARD

The single-tuple sort key from v1(`(skill_penalty, within_repeat, soft_prefer_penalty, area_repeat, -days_ago, rank)`) was lexicographic – earlier terms dominated absolutely. That made tuning impossible: you couldn't say "make rotation matter *a little* more than skill" without changing tuple order entirely.

The scorecard collapses everything into one cost. Weights are continuous, so adjustments are gradient. The lexicographic tuple is preserved as a final tiebreak: `rotation_key()` returns `(total, rank)`, which means a true tie on total falls back to roster Tie Break Rank.

Code refs: `scorecard.py:80-120` (DEFAULT_WEIGHTS), `scorecard.py:score_placement`, CHANGELOG Phase 1 / 4c notes.

ROTATION_KEY — THE SORT TUPLE.

After scoring, candidates are sorted by `rotation_key(rk, slot_code, ...)`. The function returns a 2-tuple `(total, rank)` — the scorecard total followed by roster Tie Break Rank as the deterministic tiebreak.

10.1 — SORT WRAPPER

```
def rotation_key(rk, slot_code, skill_priority=False,
                 soft_prefer_set=None, day_name=None):
    sc = score_placement(rk, slot_code,
                        skill_priority=skill_priority,
                        soft_prefer_set=soft_prefer_set,
                        day_name=day_name)
    return (sc["total"], sc["rank"])
```

10.2 — WHEN SKILL PRIORITY FIRES

Skill priority is a per-call flag, true for the constrained block where coverage doubling means a single empty card hits two zones. The set of skill-priority-enabled slots:

```
DOUBLE_COVERAGE_SLOTS = {
    "Zone1", "Zone2", "Zone6", "Zone7", "Zone8", "Zone10",
    "MRR1", "WRR1", "MRR6", "WRR6", "MRR7", "WRR7",
    "MRR8", "WRR8", "MRR10", "WRR10",
}
# Excluded: Zone 9, Z9 SR (specialist preference handles those), Trash, MP, Admin.
```

For those slots, the skill-match component is augmented with `-skill/20`, biasing toward high-skill TMs without overwhelming other components.

10.3 — PER-DAY SETUP

Three calls fire at the start of each day's fill loop:

1. `scorecard.set_current_day(iso_date_str)` — sets the day for fatigue / area-diversity windowing.
2. `scorecard.set_soft_overrides_for_day(soft_by_dn)` — per-night overrides keyed by display name.
3. Pool composition is rebuilt for the day's grave / overlap pools.

10.4 — WHY `(TOTAL, RANK)` AND NOT `(TOTAL,)`

Floating-point totals can collide exactly when two candidates have the same component values. Falling back to roster Tie Break Rank gives a deterministic, audit-traceable choice. The roster rank is fixed for the week, so two runs of the engine on the same inputs always produce the same output. **The engine is idempotent by design.**

10.5 — AUDIT TRAIL PER PLACEMENT

`_record_placement()` attaches the scorecard breakdown to the audit:

```
record["scorecard"] = {
    "total": round(sc["total"], 3),
    "fatigue_pts": sc["fatigue_pts"],
    "components": {k: round(v, 3) for k, v in sc["components"].items()},
}
```

Every row of the audit "Placement Log" carries the breakdown. When a placement looks weird, you can see exactly which component drove the choice.

OPERATIONAL TIP

If a placement consistently looks wrong, sort the audit's placement log by `scorecard.total` for the slot in question across the week. The component breakdown will tell you whether it was a fatigue penalty, a within-repeat hit, or a pair-affinity nudge that drove the choice.

Code refs: `scorecard.py:rotation_key`, `fill_engine.py:_record_placement`.

PHASE 4F – GLOBAL OPTIMUM.

The greedy fill order is fast and explainable but locally myopic. By the time the engine reaches Z8 (~15th placement of the constrained block), the pool is already depleted of strong candidates. Z8's historical fill rate hovered around 68.6% before Phase 4f. The LAP solver fixes that by treating the entire constrained block as one global optimization.

11.1 – ACTIVATION

Opt-in: set `placement_method="lap"` in the engine config (or pass via `--config-override`). When active, the daily fill calls `_lap_fill_constrained_block()` instead of the slot-by-slot greedy path for the 22-slot constrained block. Falls back gracefully if scipy isn't importable.

11.2 – COST MATRIX DESIGN

AXIS	CONTENTS
Rows	Eligible pool members for the day, padded to square with dummy rows.
Cols	22 constrained slots (10 RRs + Admin + Z9 SR + Z1 + Z4 + Z5 + Z8), padded to square with dummy cols.
Cell cost	<code>scorecard.score_placement(rk, slot_code)["total"]</code> – same scorecard the greedy path uses. Lower = better.
HARD_BLOCK_COST	<code>1e9</code> – for ineligible / accommodation-blocked / hard-avoid TMs. Solver will treat as forbidden.
SOFT_BLOCK_COST	<code>1e6</code> – for back-to-back violations. If pool is tiny, solver will still pick least-bad option.
Dummy row cost	<code>HARD_BLOCK_COST</code> for real slots; 0.0 for dummy slots. Dummy TMs never win real slots.
Dummy col cost	<code>0.0</code> for all real TMs. Real TMs may freely "win" dummy slots – they just go un-placed.

11.3 – SOLVE

Single call to `scipy.optimize.linear_sum_assignment(cost)`, which runs the Jonker-Volgenant algorithm in $O(n^3)$ over the padded matrix. Returns a row-to-col mapping that minimizes total cost across all 22 slots simultaneously.

11.4 – WHAT THIS FIXES

In the greedy engine, slots fill sequentially. Z8 sits late in the constrained-block order, so by the time it's its turn the candidate pool is thin – the engine picks a marginal candidate or leaves the slot unresolved. The LAP solver sees all 22 slots simultaneously and reserves a viable Z8 candidate during the global solve. Z8 fill rate jumps to near-100% on weeks where the pool can support it.

11.5 — WHY NOT LAP FOR THE WHOLE BOARD

The constrained block (22 slots, ~30–40 candidates) is at the sweet spot of LAP's complexity / payoff curve. Adding zones 2/3/6/7/9/10, trash, support, and overlaps would balloon the matrix to ~50×50+ and add coupling concerns (training pre-pass, overlap-pool routing) that don't fit the LAP model cleanly. Outside the constrained block, greedy is fine — pool depth is no longer the issue.

11.6 — FALLBACK PATH

Any slot where every candidate is `HARD_BLOCK_COST` is returned with `None` assignment. The caller logs an `LAP_FALLBACK` audit item with reason text, then falls back to the slot-by-slot greedy `place()` for that single slot.

11.7 — IDEMPOTENCE

The Hungarian algorithm produces a deterministic optimum for the same input matrix. As long as upstream state — pool, scorecard, accommodations — is unchanged, two runs produce the same assignment. The LAP path preserves the engine's idempotence guarantee.

Code refs: `glcr_engine/lap_solver.py`, `fill_engine.py:1081` (`_lap_fill_constrained_block`).

TRAINING PRE-PASS.

Training rules trump everything. Before any general fill logic runs, `training_prepass()` looks up the date in `Rules/Training Config.json` and force-places the trainee & trainer per the GLCR training SOP.

12.1 – CONFIG SHAPE

```
{
  "2026-04-27": {"trainee": "Cookie", "trainer": "Sheri 0", "day": 2},
  "2026-04-28": {"trainee": "Cookie", "trainer": "Sheri 0", "day": 3},
  ...
}
```

12.2 – DAY 1-6 PLACEMENT MATRIX

DAY	TOPIC	TRAINEE	TRAINER
1	Restroom	Eligible RR 1+2 (or MP if same gender)	RR 1+2 (their gender)
2	Zones	MP 1 / MP 2	Zone 1, 2, 3, or 4
3	Smoking Room	F/Sa/Su: Z9 SR · M-Th: MP	Zone 9
4	Shadow RR	Any eligible RR	Opposite-gender RR (same number) or paired Zone
5	Shadow Zone	Z1, Z2, Z3, Z4, Z6, Z7	Adjacent zone
6	ADP Courses	MP 1 / MP 2	Z1, Z2, Z3, Z4, Z6, Z7

12.3 – FORCE-PLACE MECHANICS

`place_training()` bypasses `pick()` entirely. It still runs guards:

- **Slot-not-in-template** – emits `TRAINING_SLOT_NOT_IN_TEMPLATE` warning, skips the placement.
- **TM already placed today** – emits `TRAINING_CONFLICT` warning, skips.
- **Eligibility** – added 5/3/26: even training won't put a TM in an N slot. Emits `TRAINING_TM_INELIGIBLE` warning, skips.

If the placement succeeds, the slot is added to `filled_slots` – locking it against subsequent grave fill – and recorded as a training-typed placement in the archive.

12.4 – DAY 3/4 SWAP

Training Days 3 (smoking) and 4 (shadow restroom) may be swapped if the pool needs it. The supervisor handles by editing `Training Config.json`; the engine picks up the new day number and applies the corresponding placement rules. No special engine logic.

12.5 – TRAINEE DETECTION

Trainees are auto-derived as TMs with `skill_score ≤ 3` . Currently:

```
TRAINEE_DISPLAY = {"Seth", "Trenidee"}
```

Outside the pre-pass, both are filtered from `pick()` (because `skip_trainees=True` is the default), preventing them from accidentally landing in a zone.

Code refs: `fill_engine.py:1261-1287` (place_training, training_prepass), `rules.md §13-§14` .

AUDIT PIPELINE.

Every fill, audit, and reconcile run produces an audit. Three artifacts: a JSON for tooling, a markdown for humans, and an append-only [Grave Placement Archive.xlsx](#) for cross-week rotation tracking. The audit is the contract between the engine and the supervisor.

13.1 – SEVERITY LADDER

LEVEL	MEANING
CRITICAL	Z9 area uncovered, write-verification failed, >0 headcount delta. Goes to top of audit's ## ACTION REQUIRED block.
ERROR	Eligibility violation, trainee violation, duplicate placement, unrostered TM, template mismatch, unresolved 1st-priority slot.
WARNING	Display name mismatch, headcount short, overlap anomaly, hard-preference overridden, sweeper unassigned, RR half-staffed.
INFO	Tie-break resolved, PM/AM overlap used, slot inactive on day, skill-stretch placement.

13.2 – AUDIT JSON SHAPE

```
{
  "week_ending": "2026-04-30",
  "week_start": "2026-04-24",
  "run_timestamp": "2026-04-23T22:14:01Z",
  "mode": "fill",
  "anchor_cell": "A3",
  "summary": {
    "total_fill_slots": 154,
    "filled": 148,
    "unfilled": 6,
    "errors": 0,
    "warnings": 2
  },
  "placements": [
    {
      "date": "2026-04-26",
      "zone_slot": "Zone8",
      "priority": "Zone",
      "tm_display_name": "Daryl",
      "pool_type": "grave",
      "tm_skill": 7.5,
      "slot_difficulty": 6,
      "stretch": false,
      "scorecard": {
        "total": 0.612,
        "fatigue_pts": 8,
        "components": { "skill_match": 0, "within_repeat": 0,
          "area_diversity": 1, ... }
      }
    },
    ...
  ],
  "audit_items": [...]
}
```

13.3 – HEADCOUNT RECONCILIATION

For every date: **scheduled_count** (unique TMs across grave + PM OL + AM OL) vs **placed_count** (unique TMs on the board). Delta of **0** is clean; **< 0** warns; **> 0** is critical (logic error – duplicate or wrong-date pool).

13.4 – POST-FILL VALIDATION

Before declaring the fill complete, the engine reads back every written cell to confirm it contains the expected value, scans for duplicates across the full board, confirms eligibility for every placed TM, and re-checks trainee restriction. Failures here are logged as errors and surfaced before the file is saved.

13.5 – BACKEND LOGGING

After every fill / live-update, the engine also emits a backend event:

```
log_event(
    event_type="dispatch",
    event_date=fill_date,
    shift="graves",
    title=f"Deployment fill - week ending {week_ending}",
    summary=f"{n_placements} placements, {n_unresolved} unresolved, "
            f"{n_audit_critical} critical",
    metadata={"kind": "fill", "week_ending": ...,
            "engine_version": "v8", ...}
)
```

And, for callouts and `Z9_AREA_UNCOVERED`, a `capture(content_type="flag", sentiment="flag", ...)`. The dashboard timeline reads from these events; the search index reads from the captures.

13.6 — RECONCILE MODE

When the supervisor manually edits the workbook and asks for an audit without refilling, the engine reads the current state, compares every non-blank cell to what the automated fill would have produced, and categorizes deviations as *correct manual fix*, *potential error*, or *unexplained blank*. Reports without rewriting; flags `mode: reconcile` in the audit.

Code refs: `fill_engine.py` audit emit, `audit-spec.md` (full taxonomy), `SKILL.md §Live-Update Mode`.

MID-WEEK CHANGES.

When a TM calls off mid-week, gets swapped, or is pulled for a special task, the engine has to keep three artifacts in sync: the filled workbook, the placement archive, and the rendered deployment book. Skipping the archive update silently breaks future-week rotation tracking.

14.1 – TRIGGERS

- "X called off"
- "Move Y from A to B"
- "Z is doing special task today"
- "Swap A and B"
- "Put Z in slot today"

14.2 – WORKFLOW

1. Restate the change, get confirmation.
2. Update the filled workbook (cell edit, save).
3. Update the placement archive (source of truth for cross-week rotation).
4. Re-render the deployment book.
5. Analyze downstream impact — does rotation logic shift for remaining days?
6. Surface a re-deploy suggestion to the supervisor.
7. Log to backend (`log_event` with `kind: live_update`).
8. For callouts, also `capture` a flag with `kind: callout` .

14.3 – ROTATION IMPACT

Three scorecard components depend on archive state and are *re-evaluated* for the rest of the week after a live update:

COMPONENT	WHAT CHANGES
<code>within_repeat</code>	Removing TM X from a slot earlier this week unlocks them as a candidate for that slot later this week.
<code>area_diversity</code>	If a TM's actual area shifts, tomorrow's <code>area_repeat</code> penalty fires off the new area.
<code>cross_week_rotation</code>	Updates "days_ago" for the affected slot; future weeks see correct freshness.

14.4 – SWEEPER RE-ASSIGNMENT

If the male who was going to do a sweeper route calls off, the auto-assignment may shift to a different male. The skill re-runs sweeper assignment as part of the live-update.

14.5 — RE-DEPLOY SUGGESTION

If today is mid-week and the change affects rotation for future days, the engine surfaces a suggestion: *re-deploy all remaining days, just specific affected days, or accept the gap*. The supervisor decides. Partial re-deploy locks past days (already happened) and regenerates only the future portion using the updated archive.

WHY ARCHIVE UPDATES MATTER

Without an archive update on a live edit, the engine would still see the original placement when computing next week's rotation. Sheri O moved off Admin Tuesday wouldn't get her "fresh on Admin" credit, and the engine would over-place the Admin rockstar pool the following week.

Code refs: [SKILL.md §Live-Update Mode](#), [rules.md §17](#).

WORKBOOK, ARCHIVE, BACKEND.

The engine writes to three persistence layers. Each has its own role; together they make the deployment auditable and the rotation reproducible across weeks.

15.1 — FILLED WORKBOOK

Working copy of the master template, named `Week Overview - Filled - [week-ending-date].xlsx`. Lives at `Outputs/[week-ending]/`. The engine writes only to named ranges (`day{Weekday}-{SlotCode}-{Number}`); never modifies the canonical blank.

15.2 — PLACEMENT ARCHIVE

Append-only `Grave Placement Archive.xlsx`. One row per placement. Schema:

COLUMN	CONTENT
Week Ending Date	ISO Thursday of the deployment week
Calendar Date	Specific grave date of the placement
Day of Week	Friday – Thursday
Section	Zones / Restrooms / Trash-Admin-MP
Slot, Named Range, Cell	e.g. "Zone 3", <code>dayMondayZone3</code> , <code>D6</code>
TM Display Name, Employee Name	Board-ready + roster legal name
Pool Type	<code>grave</code> / <code>pm_overlap</code> / <code>am_overlap</code> / <code>training</code>
Tie Break Rank	From roster col F
Run Timestamp, Notes	ISO datetime + audit flags / tie-break notes

15.3 — AUDIT JSON + MARKDOWN

Co-located with the filled workbook at `Outputs/[week-ending]/Grave Deployment Audit - [date].{json,md}`. The audit always references the workbook it audited.

15.4 — GLCR MEMORY BACKEND

Two write paths from the engine:

TOOL	WHEN	EFFECT
<code>log_event</code>	Every fill / live-update	Dashboard timeline event with metadata <code>{kind, week_ending, engine_version, ...}</code>
<code>capture</code>	Callouts, <code>Z9_AREA_UNCOVERED</code>	Searchable flag with <code>content_type=flag</code> , <code>sentiment=flag</code> , entity refs

And one read path: `recent(content_type=flag, since_days=14, entities=[id])` for the recent-flag soft penalty in scoring.

15.5 – WHERE EACH LAYER WINS

WORKBOOK

Source of truth for "what we're doing tonight." Edited live by the supervisor; consumed by the renderer.

ARCHIVE

Source of truth for "what we did across weeks." Drives `cross_week_rotation`.

BACKEND EVENTS

Source of truth for the event stream – when the dispatch ran, what mode, what artifacts. Powers the dashboard timeline.

BACKEND CAPTURES

Source of truth for "what's been flagged about this TM or this date." Powers the search and the flag-penalty.

15.6 – MIGRATION TO SUPABASE (5/5/26)

Roster, profiles, slot difficulty, training schedule, overlap tasks, scorecard config, and engine overrides all live in Supabase as of 5/5/26 (see `shared.db` imports at the top of `fill_engine.py`). The legacy `Rules/*.json` files are kept on disk as a fallback but no longer read by default. The workbook + archive remain file-based output artifacts.

Code refs: `fill_engine.py:73-95` (DB imports), `workbook-contract.md §5-§6`, `SKILL.md` backend tools table.

CONFIGURATION FILES.

Knobs the engine reads at startup. Anything not documented here is hard-coded and intentional. If a change needs to be made, prefer touching one of these over editing engine code.

16.1 — ELIGIBILITY ROSTER (EXCEL)

`Rules/Eligibility Roster.xlsx` — 51 active TMs, full Y/N matrix per slot. Source of truth for who can do what; if migrating, the DB equivalent is `get_engine_roster_from_db()`.

16.2 — TM PROFILES

Per-TM `{skill_score, preferences[], accommodations[], pair_affinities[], score_history[]}`. Owned by the `glcr-tm-profiles` skill. Never edit directly.

16.3 — SLOT DIFFICULTY

Per-slot integer 1-10. Drives `skill_match` and `override_prefer_easier` (threshold 6).

16.4 — SLOT LOAD SCORES

Per-slot integer 1-10 representing physical / cognitive demand. Drives `fatigue_index` and `weekly_load_balance`; `override_avoid_high_load` uses threshold 6.

16.5 — TRAINING CONFIG

`{ISO_DATE: {trainee, trainer, day: 1-6}}`. The engine reads this at startup; entries with a date matching the fill window fire the training pre-pass.

16.6 — OVERLAP TASKS

Canonical `{PMOL1: ..., AMOL1: ..., _per_day_overrides: {...}}`. Read by both engine (audit metadata) and renderer (display).

16.7 — UTILITY PORTERS

Per-day `{ISO_DATE: [{role, name}, ...]}`. Up to 8 entries per date.

16.8 — SCORECARD WEIGHTS

Per-component float weights. Defaults set in `scorecard.DEFAULT_WEIGHTS`; override via DB `get_scorecard_config()` or `--config-override` JSON.

16.9 — ENGINE OVERRIDES (PER-NIGHT, PER-TM)

Three soft override types set per night via `set_soft_overrides_for_day()` :

OVERRIDE	EFFECT	DEFAULT WEIGHT
<code>prefer_easier</code>	Penalty in slots with difficulty > 6	1.5
<code>avoid_high_load</code>	Penalty in slots with load > 6	1.5
<code>priority_placement</code>	Slot-agnostic <code>-1</code> bonus	1.0

16.10 — MANUAL RENDERER OVERRIDES

Two in-script dicts in `render_deployment_book.py` :

```
MANUAL_ALERTS = {
    # "2026-04-28": {"aux_mp_1": "ADP Training"}
}

SWEEPER_REASSIGNMENTS = {
    # "2026-04-29": {"sweeper_9_10_sr": "rr_8"}
}
```

16.11 — PHASE 4C THRESHOLDS

Two configurable thresholds previously hard-coded:

THRESHOLD	PURPOSE	DEFAULT
<code>override_difficulty_threshold</code>	Slots above this are "hard" for <code>prefer_easier</code>	6
<code>override_load_threshold</code>	Slots above this are "heavy" for <code>avoid_high_load</code>	6

Code refs: `scorecard.init`, `shared/db.py`, `workbook-contract.md §9`.

GUARDRAILS & TIE-BREAKS.

Catalog of known edge cases and how the engine handles each. These are operational invariants — most are baked into `pick()` and `place()`, all are surfaced in the audit when triggered.

17.1 — ENGINE GUARDRAILS (ALWAYS-ON)

- **Never edit the canonical blank template** — the engine works on a copy.
- **Never force an illegal placement** to make the board look complete.
- **Never fill 2nd / 3rd priority slots** if any 1st slot on that date is unresolved.
- **Never place Seth or Trenidee outside MP / Support** — trainee restriction.
- **Never place Daryl in his accommodation-blocked slots** (Z7, Z8, MRR 1+2, WRR 1+2).
- **Never silently normalize a display-name mismatch** — flag in audit.
- **Never duplicate a person on the same date** across any board section.
- **Always generate an audit** — even for partial or manually-edited boards.
- **Always flag `Z9_AREA_UNCOVERED`** as critical when both Z9 and Z9 SR are empty.
- **Always re-render the deployment book** after fill or manual edit.
- **After any live-update:** update workbook AND archive AND log to backend.
- **After any live-update:** surface downstream impact analysis.

17.2 — TIE-BREAK LADDER

When two candidates score identically on the rotation key:

1. `scorecard.total` rounded to floating-point precision.
2. Roster `Tie Break Rank` (lower wins).
3. Audit emits `TIE_BREAK_RESOLVED` info item documenting the choice.

17.3 – SPECIFIC EDGE CASES

CASE	ENGINE BEHAVIOR
Personal-Monday overlap with calendar Monday	Defer to human; emit <code>PERSONAL_MONDAY_TIE_BREAK</code> warning.
TM in schedule but not on roster	Do not place; emit <code>UNROSTERED_TM_IN_SCHEDULE</code> .
Roster Y but TM not on schedule	TM not available; do not place.
AM overlap TM also grave-scheduled previous night	Count once; do not double-place.
Rule sheet references zone not in template	<code>TEMPLATE_MISMATCH</code> ; never create new cells.
"Prefer" elig has no eligible TM	Fill with next best eligible; never leave preference-empty.
Z9 SR specialist preference scope	Only Fri/Sat. Sun–Thu rotates; Z9 (the floor zone) never uses it.
Sweeper has no eligible male	Route unassigned; <code>SWEeper_UNASSIGNED</code> warning.
RR with one side empty	Render-time warning. Engine TBD; current behavior allows half-staffed.
Display-name normalization mismatch	Use roster name; emit <code>DISPLAY_NAME_MISMATCH</code> warning.
Hard-pref avoid empties pool	Override; emit <code>HARD_PREFERENCE_OVERRIDDEN</code> warning per overridden TM.
Back-to-back guard empties pool	Override silently; back-to-back is a guideline, not a rule.
LAP solver can't resolve a slot	Returns <code>None</code> ; caller emits <code>LAP_FALLBACK</code> ; greedy <code>place()</code> fills the slot.

17.4 – KNOWN TEMPLATE ANOMALIES

Worth flagging when the engine touches them:

- `dayMondayPM1-5` may reference `#REF!` in some Excel versions — verify before Monday PM placements.
- `dayThursdayWRR6/7/8` historically pointed to the same cell `I19` — verify Thursday womens RR placements manually.
- `dayMondayMRR9` defined as `F16` — same cell as `dayMondayMRR10`. If both fill, conflict exists.
- `key_wo_d1_date` (`A3`) must be set to the correct Friday before a run — only manual cell.

Code refs: `fill_engine.py` guardrails in `place()`, `SKILL.md` `$Guardrails`, `workbook-contract.md` `$7`.

TERMS.

A reference for terminology that's specific to this engine. When a term has a precise meaning in the codebase, that meaning is what you'll find here.

ANCHOR DATE

The Friday in cell `A3` of the Week Overview workbook. All other dates cascade from it. The only manually-edited cell.

AUDIT ITEM

One JSON object in the audit's `audit_items[]` array. Has a `severity`, `type` (taxonomy code), `tm_name`, and `detail` string.

BACK-TO-BACK GUARD

Hard rule: a TM cannot be placed in the same exact zone or RR side as yesterday. Has a fallback so the slot doesn't go empty.

CONSTRAINED BLOCK

The 22 slots filled by the LAP solver path: 10 RRs + Admin + Z9 SR + Z1 + Z4 + Z5 + Z8.

DISPLAY NAME

Board-ready TM name (typically first name or roster-canonical form). Distinct from Employee Name. *Always* use display name on the board; flag mismatches.

ELIGIBILITY

`Y` / `N` per (TM, slot) in the roster. `Y` = placeable. `N` = forbidden by roster. Cannot be overridden.

FILL ORDER

The sequence of solve passes per day: training pre-pass → restrooms → admin → Z9 SR → zones (skip-priority) → trash → support → overlaps.

HARD BLOCK

A blocker that filters a TM from the candidate pool before scoring. Sources: roster `N`, hard accommodation, hard avoid preference, slot-pref avoid, trainee filter.

LAP

Linear Assignment Problem. Solved with the Hungarian algorithm via `scipy.optimize.linear_sum_assignment`. Optional fill path for the constrained block.

MUST-FILL ZONE

Z1, Z4, Z5, Z8. Never intentionally skipped. Empty must-fill is a critical audit.

PAIR AFFINITY

Soft scoring component for TMs with explicit "prefer to work near X" / "avoid working near X" preferences. Walks adjacent + same-area co-occupants.

PERSONAL-MONDAY

A non-standard work week where a TM's "Monday" lands mid-week. Triggers placement on their personal Monday rather than calendar Monday.

POOL

One of `grave`, `pm_overlap`, `am_overlap`, `training`. Not all pools are eligible for all slots.

RECONCILE MODE

Audit the workbook after manual edits without rewriting. Categorizes deviations as correct fix / potential error / unexplained blank.

ROCKSTAR (ADMIN)

Sheri O, Sherry B, Jamie, Tawnya. The four traditional Admin preferred TMs. Soft preference (`soft_prefer_set`), not hard.

SKILL PRIORITY

Per-slot flag activating an additional `-skill/20` term in the scorecard for double-coverage zones & restrooms.

SKILL STRETCH

A placement where `(slot_difficulty - tm_skill) > 2`. Logged in audit as info-level.

SKIP PRIORITY

The order zones empty when the pool runs short: Z9 → Z6 → Z7 → Z2 → Z3 → Z10. Z1, Z4, Z5, Z8 never skipped.

SOFT OVERRIDE (PER-NIGHT)

`prefer_easier` / `avoid_high_load` / `priority_placement`. Set per-TM per-night via `set_soft_overrides_for_day()`.

SPECIALIST (Z9 SR)

Joy, Melissa, Mike S, Silvia. Hard preference for Z9 SR on Fri / Sat. Drops to standard rotation Sun – Thu.

TIE BREAK RANK

Roster integer 1-51. Final tiebreak in `rotation_key`. Lower wins. Stable across the week.

TRAINEE

TM with `skill_score ≤ 3`. Auto-derived. Locked to MP / Support slots only.

WITHIN-REPEAT

Score component: `1` if TM was placed in this exact slot earlier this week. Encourages spread across the week.

Z9 SR BUDDY

v8 conditional slot in column I, active Fri / Sat. First overflow target for unplaced grave-pool TMs on weekends.

"The board fills in this exact order every day. Sequencing from most-constrained pool to least-constrained, with overflow at the end."

— BRIAN KILLIAN, 4/28/2026

END OF DOCUMENT · GLCR PLACEMENT ENGINE v8